Chapter 02: Overview of Java Servlet and JavaServer Pages (JSP)

Department of Software Engineering
Faculty of Information Technology (HCM-UTE)
Course ID: WEPR330479
Lecturer: Msc. Khoa Hoang

Content

- Servlet
- JavaServer Pages
- MVC model
- HTTP Request, HTTP Response
- Cookie
- Session

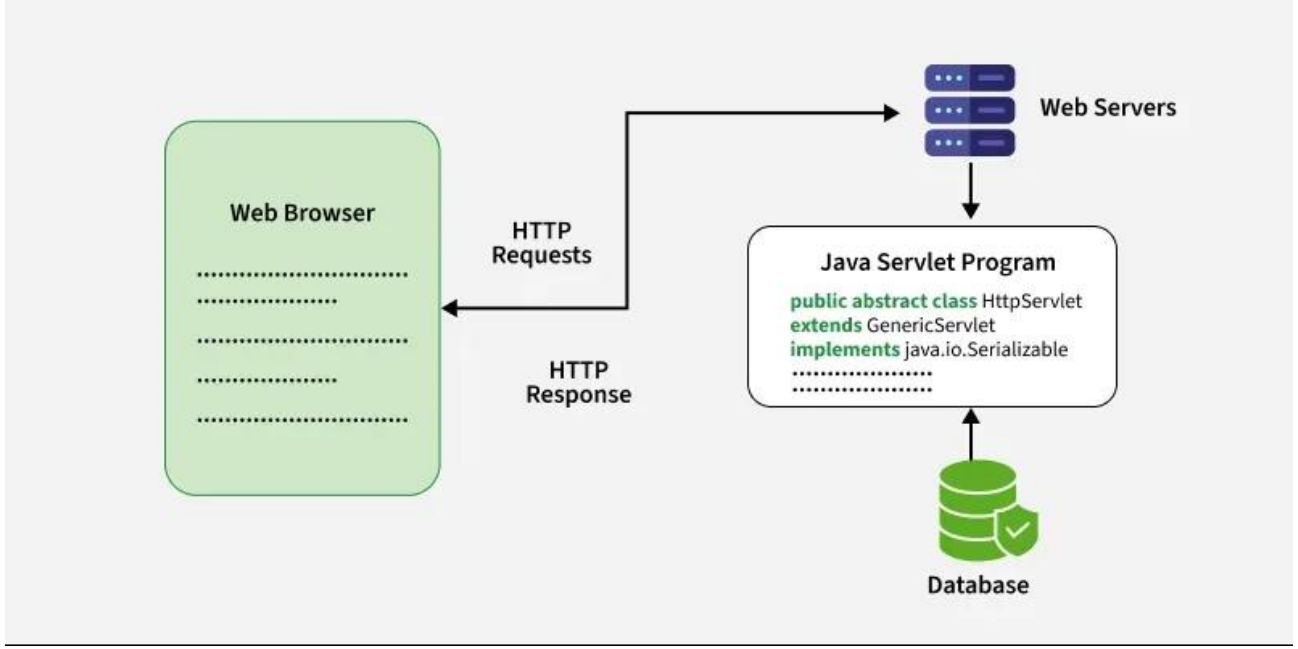
Servlet

History

- 1996: Servlet 1.0 released by Sun Microsystems, allowing server-side Java code execution.
- 1997: JDBC 1.0 introduced, giving Java apps a standard way to talk to relational databases. Servlet 2.0 added standard HTTP methods.
- 1999: JSP 1.1 introduced to let developers write HTML mixed with Java code, compiling down into Servlets.

Servlet

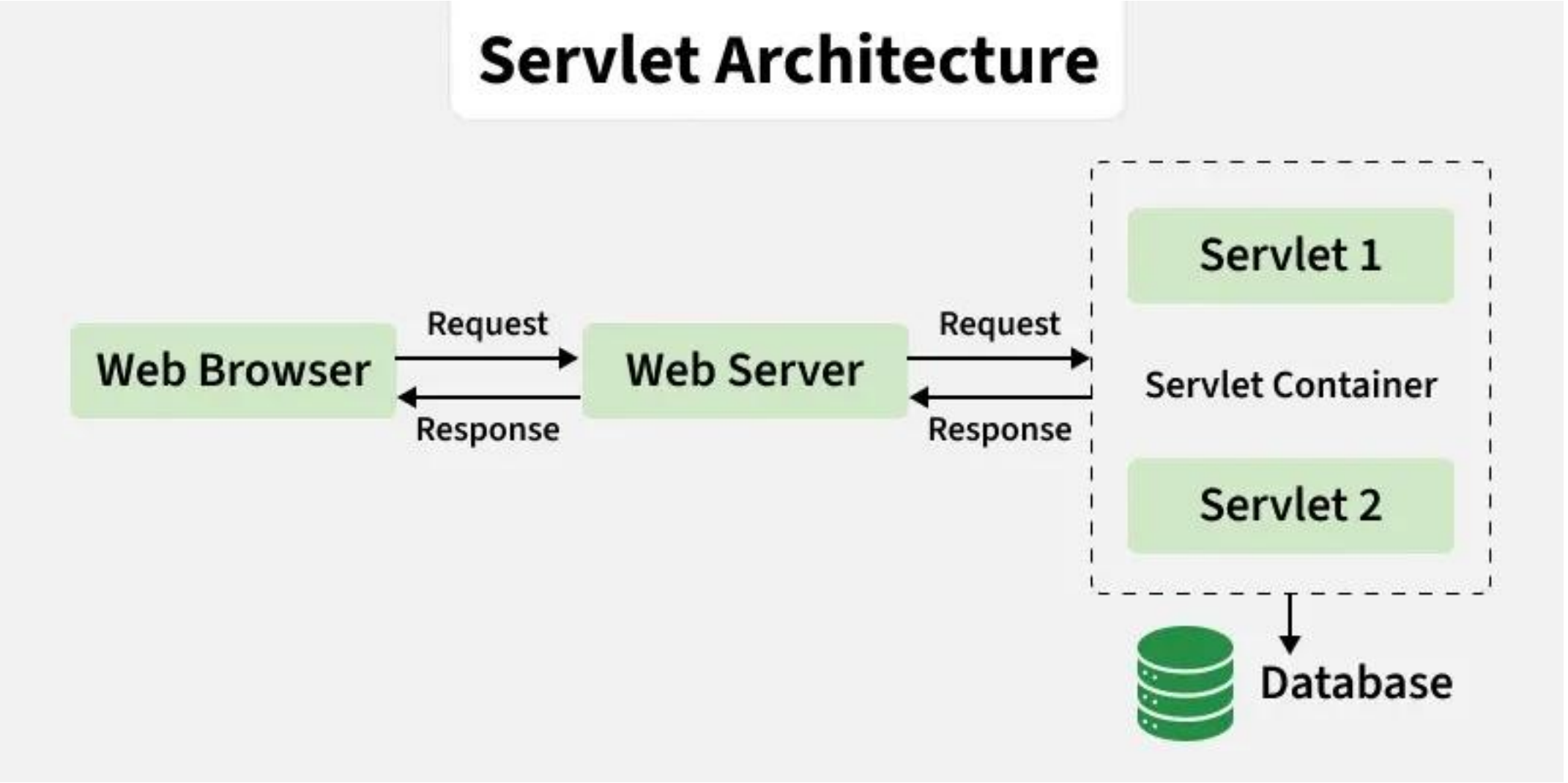
- A servlet is a server-side Java program that handles client web requests and generates dynamic responses.
- It runs inside a web container (like Apache Tomcat) and forms the core foundation for building scalable web applications and enterprise frameworks in Java.



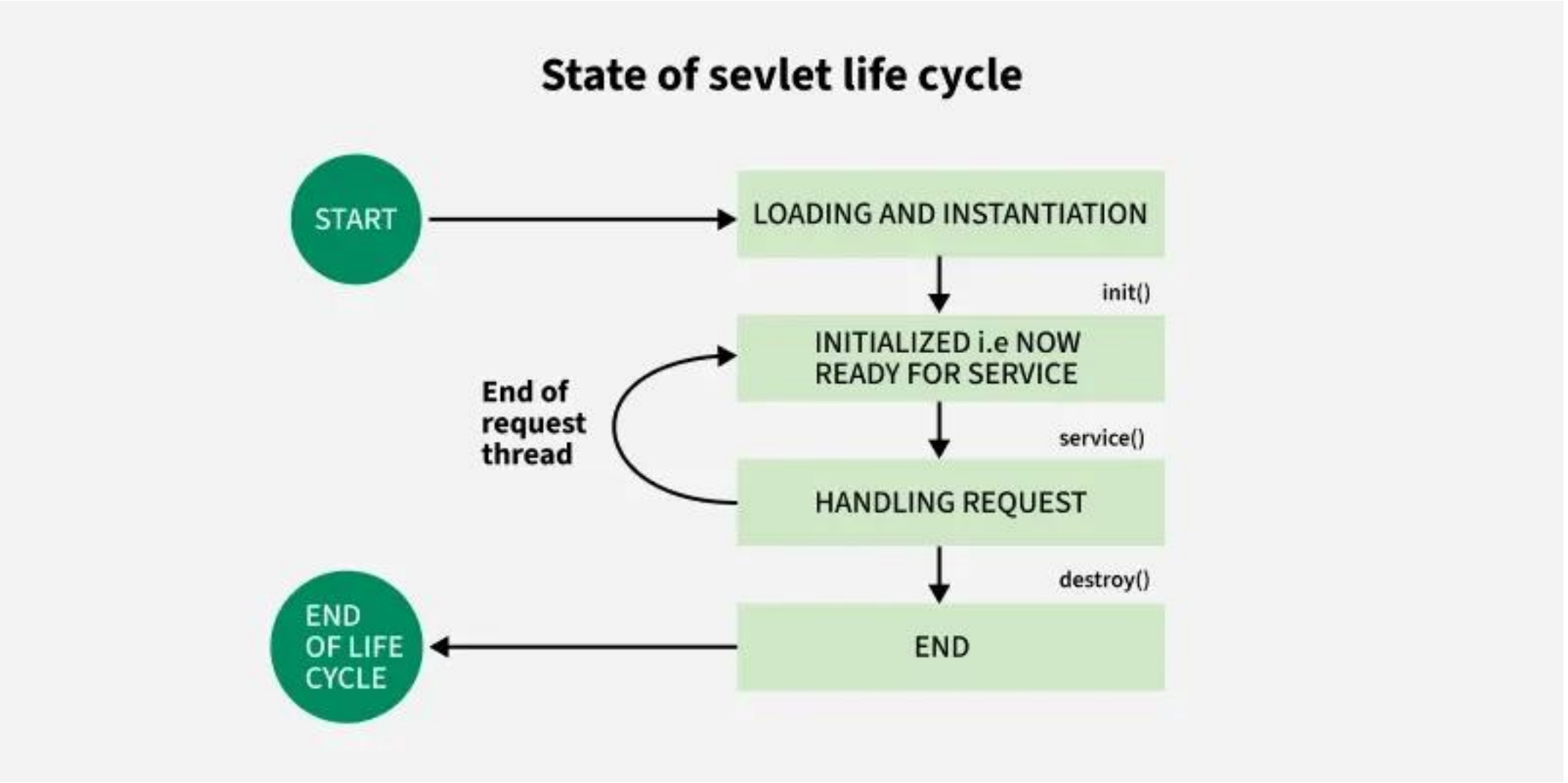
Servlet key features

- Request-Response model: processes incoming HTTP requests via methods like `doGet()` or `doPost()` and sends back data.
- Lifecycle management: managed entirely by a servlet container handling startup, request processing, and destruction.
- Thread safe & fast: reuses threads for multiple requests instead of spawning a new operating system process per request.

Servlet architecture



Servlet lifecycle



Example 1

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {

   public void init() throws ServletException {
      // Initialization code
   }

   protected void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {

      response.setContentType("text/html");
      PrintWriter out = response.getWriter();

      out.println("<h1>Hello, Welcome to Servlet</h1>");
   }

   public void destroy() {
      // Cleanup code
   }
}
```

Example 2

```java
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.io.PrintWriter;

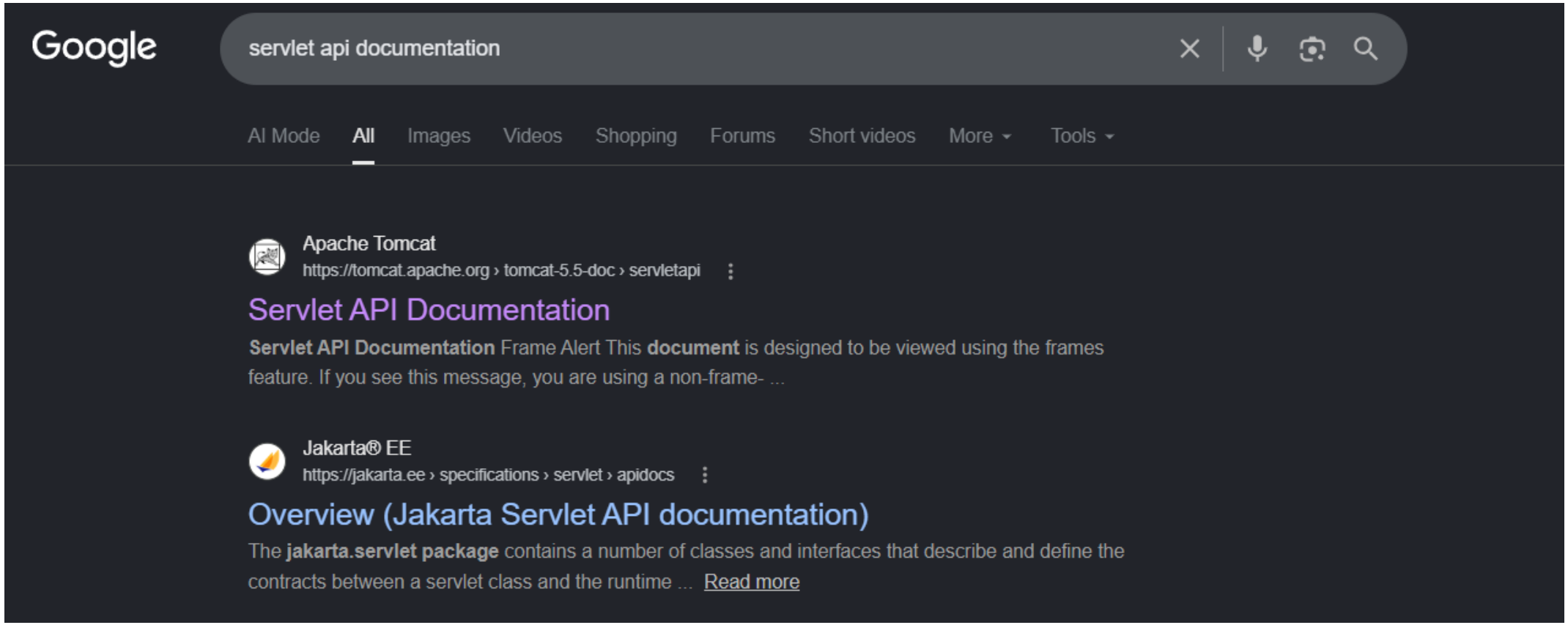
public class Servlet1 extends HttpServlet {

    protected void
    processRequest(HttpServletRequest request,
                   HttpServletResponse response)
        throws ServletException, IOException
    {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet Servlet1</title>");
            out.println("</head>");
            out.println("<body>");

            // Creating a string to store the name
            String name = request.getParameter("name");
            out.println("<h1> Hello, welcome to " + name
                        + " </h1>");
            out.println(
                "<h1><a href =\"servlet2\">Go to Servlet2</a></h1>");
            // Creating a cookie
            Cookie c = new Cookie("user_name", name);
            response.addCookie(c);

            out.println("</body>");
            out.println("</html>");
        }
    }
}
```

Servlet API documentation



https://tomcat.apache.org/tomcat-5.5-doc/servletapi/index.html

JavaServer Pages (JSP)

JavaServer Pages (JSP)

- This is a technology developed by Sun Microsystems that allows developers to create dynamic web pages by embedding Java code directly into HTML interfaces.
- Servlets are powerful for handling logic but extremely cumbersome for generating HTML, requiring you to use `out.println()` line by line.
- This approach is messy, difficult to maintain, and unfriendly to UI designers.
- There was a need for a tool that allows writing HTML naturally while embedding Java logic only when necessary.
- JSP was created to separate the presentation layer from the logic layer, making it easier for front-end and back-end teams to collaborate.
- Since JSPs are compiled into Servlets behind the scenes, they retain the full power of Servlets.

JSP syntax – JSP scriptlet

- With JSP scriptlets, you can write any Java statement, such as declaring variables or methods, initializing variables, and printing data to the browser.
- Syntax: `<% Java code here %>`
- Example
  - `<% String message = ‘hello’; %>`
  - `<% out.print(message); %>`

JSP syntax – JSP declaration

- With JSP declarations, you can declare variables and methods using Java code within a JSP file.
- You must declare variables and methods before using them in the JSP file.
- Syntax: `<%! declaration; declaration++… %>`
- Example
  - `<%! List<String> list = new ArrayList<String>(); %>`

JSP syntax – JSP expression

- A JSP expression is used to print a string of characters.
- Syntax: `<%= expression %>`
- Example 1
  - `<%! int a = 5; %>`
  - `<%= “a = “ + a; %>`
- Example 2
  - `<p>Today's date: <%= (new java.util.Date()).toLocaleString()%><p>`

JSP syntax – comment

- Syntax: <% -- JSP Comment --%>

JSP directives

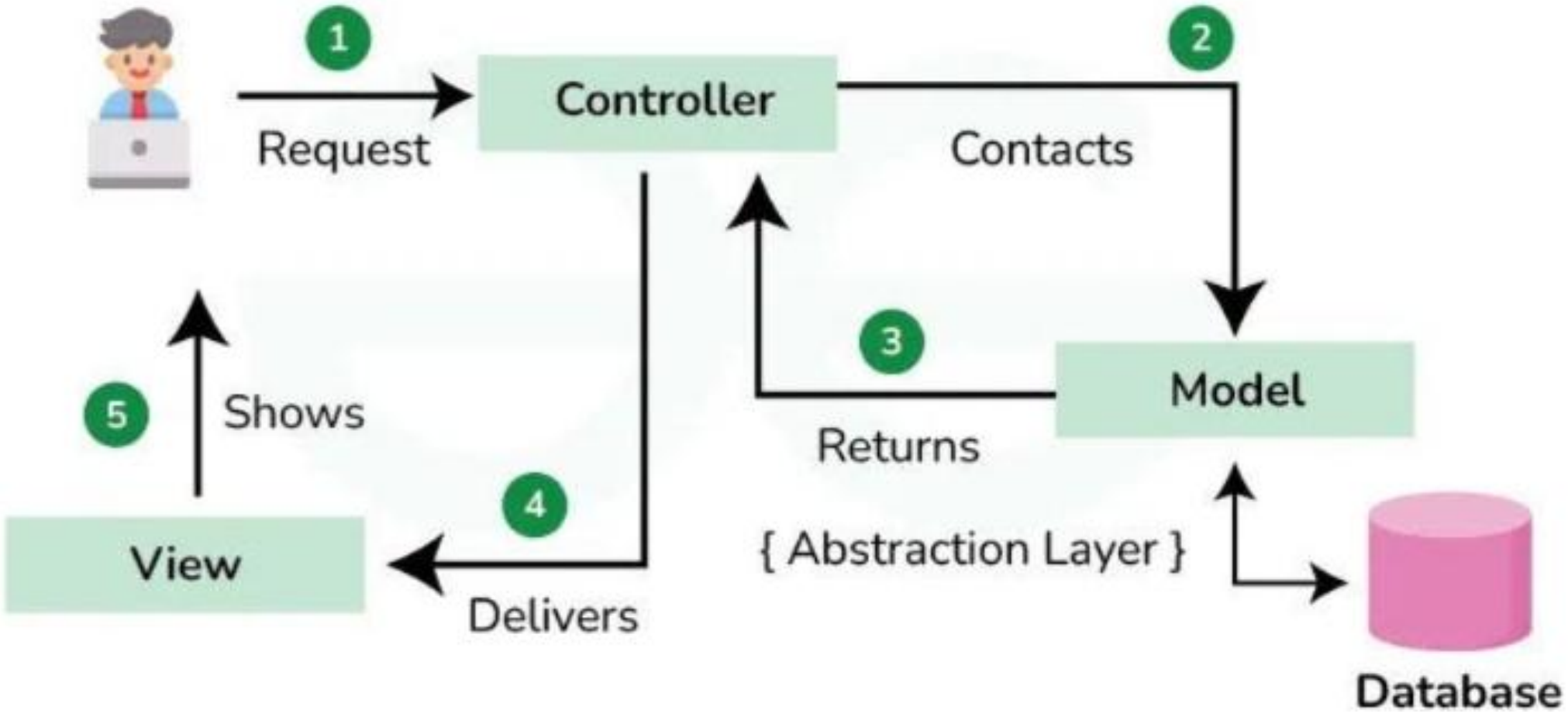
- These are directives sent to the JSP container (e.g., Apache Tomcat) at page translation time to provides information and instructions on how the container translates JSP pages into Servlets.

- Syntax: `<%@ directive attribute="value" %>`

- Three main types of directives:
  1. page: set attributes for the entire JSP page (e.g., import, errorPage).
  2. include: embed the static content of another file into the current page.
  3. taglib: declare a custom tag library (e.g., JSTL) for use in the page.

MVC model

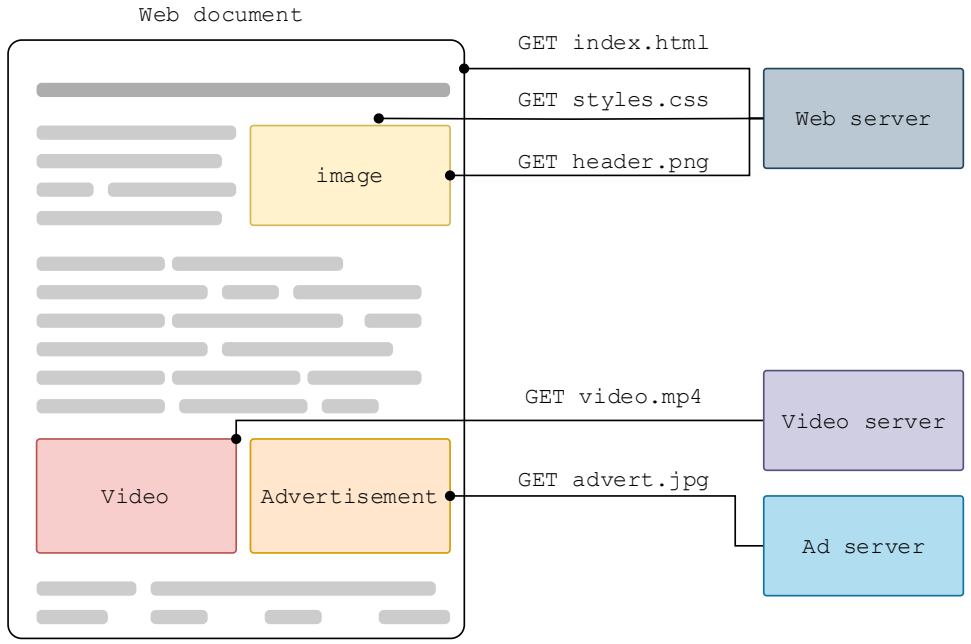
MVC model


1
Request
Controller
2
Contacts
3
Returns
Model
{ Abstraction Layer }
Database
4
Delivers
View
5
Shows

HTTP Request – HTTP Response

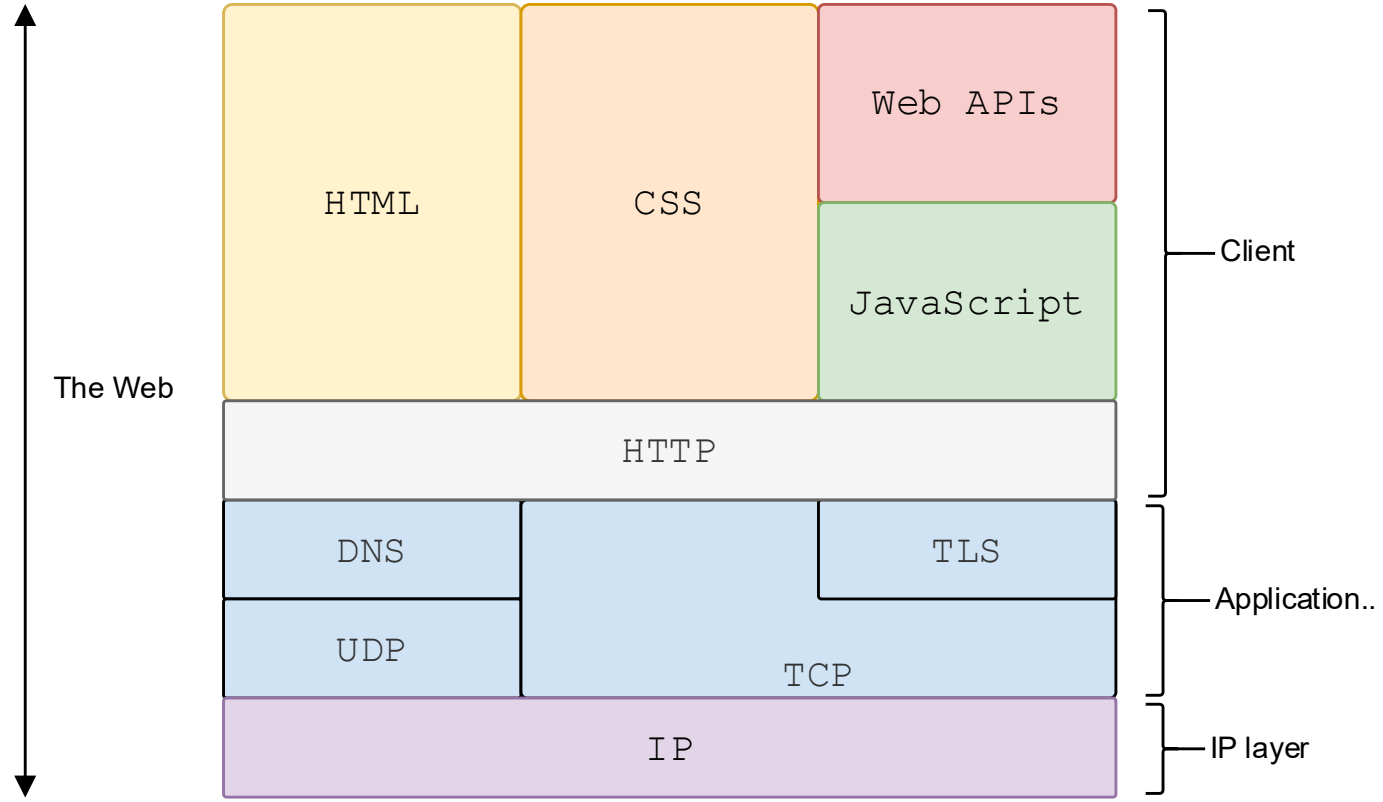
HTTP Request – HTTP Response #1

- HTTP (HyperText Transfer Protocol) is a protocol for fetching resources such as HTML documents.
- It is the foundation of any data exchange on the Web and it is a client-server protocol, which means requests are initiated by the recipient, usually the Web browser.

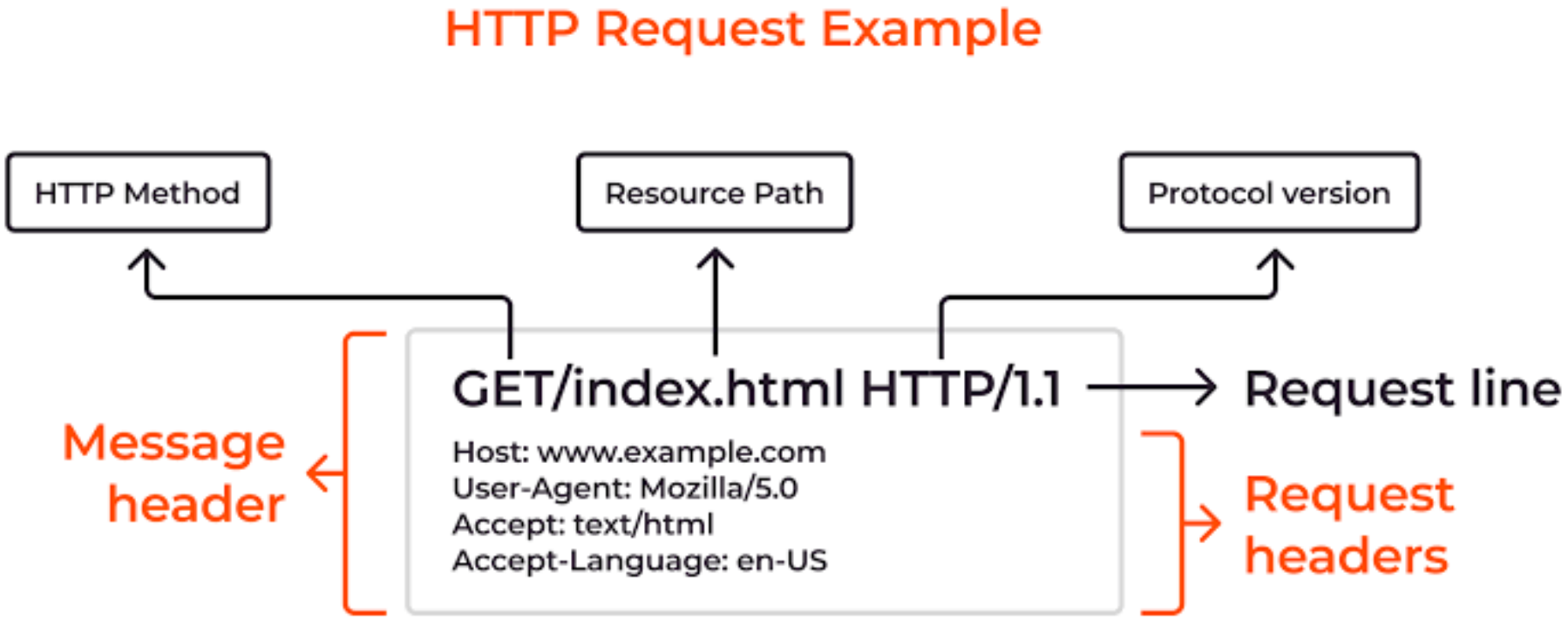


HTTP Request – HTTP Response #2

- Clients and servers communicate by exchanging individual messages.
- The messages sent by the client are called **requests** and the messages sent by the server as an answer are called **responses**.



HTTP request structure



HTTP request - GET

- The query string will be appended to the HTTP request URL.
- Example: https://example.com/?question=“abc"&category=“def“
- GET requests can be cached, bookmarked, and saved in browser history but have a length limitation (URL length is limited).
- Note: You should NOT use GET requests for sensitive data; it should only be used to retrieve data and offers no security.

HTTP request - POST

- The query string will be sent in the message body of the HTTP request.

- POST requests cannot be cached, bookmarked, or saved in browser history, and they have no length restriction.

HTTP request – Other methods

- HEAD: similar to GET but only returns the HTTP headers (no response body).
- PUT: uploads a representation of a resource to a specified URL.
- DELETE: deletes a specified resource.
- OPTIONS: returns the HTTP methods that the server supports.
- CONNECT: converts the HTTP request connection into an HTTP tunnel (typically used for HTTPS via proxy).

HttpServletRequest #1

- https://tomcat.apache.org/tomcat-5.5-doc/servletapi/index.html

- In Java Web development, every HTTP request sent to the server is encapsulated by the container (such as Apache Tomcat) into a Java object. That object is the `HttpServletRequest`.

- `HttpServletRequest` is an interface belonging to the `javax.servlet.http` package.

- The container creates an object that implements this interface and passes it to Servlet methods (such as `doGet()` and `doPost()`) for us to process.

HttpServletRequest #2

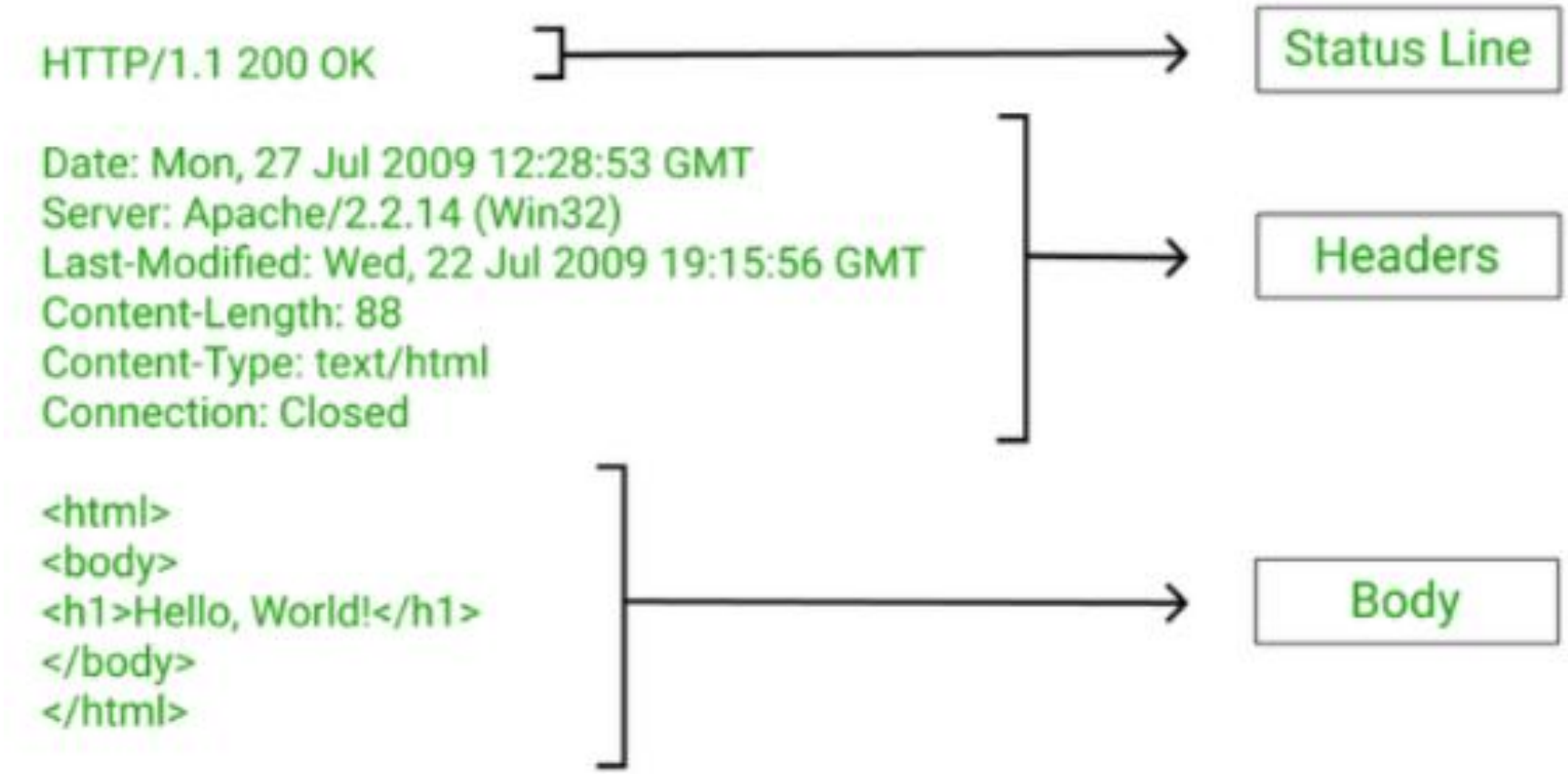
- HttpServletRequest provides developers with access to all information contained in the HTTP request.

- HttpServletRequest serves as a bridge between the HTTP world (text, headers, URLs) and the Java world (objects, methods, attributes).

- Example: https://example.com/?question="abc"&category="def"
  - `String question = request.getParameter("question");`
  - `String category = request.getParameter("category");`

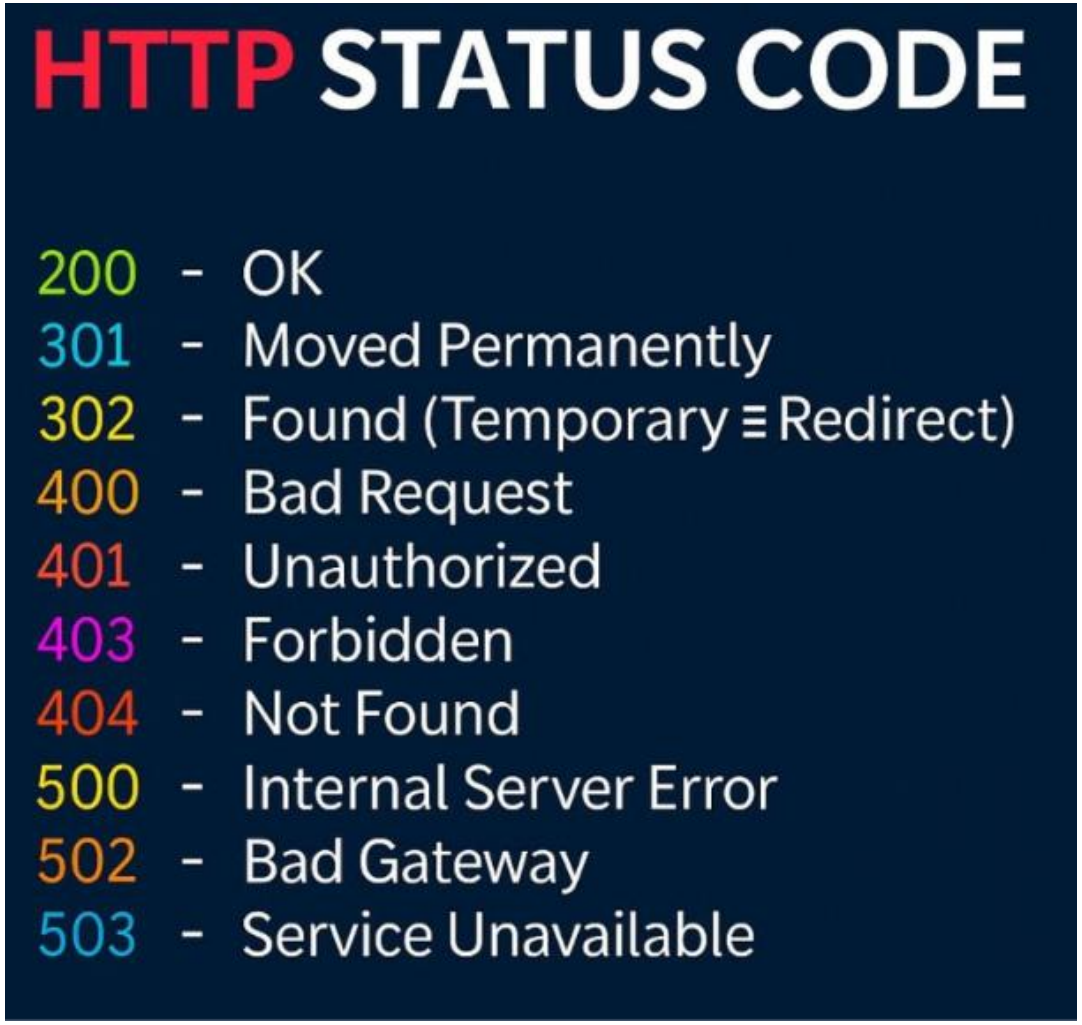
HTTP response

- An HTTP response is the result returned by the server to the client.



HTTP response – status codes

1XX Informational codes	The server has received the request and is currently processing it
2XX Success codes	The server processed the request after successfully receiving and understanding it
3XX Redirection codes	The server has received the request, but it is being redirected to another location. In some rare instances, additional actions beyond a redirect may be required
4XX Client error codes	The server was unable to locate or access the page or website. This issue originates from the site's end
5XX Server error codes	The client submitted a valid request, but the server was unable to fulfill it



https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status

HttpServletResponse

- https://tomcat.apache.org/tomcat-5.5-doc/servletapi/index.html

- How do we send a response back to the client?
- Use HttpServletResponse

- Example
  - response.setStatus(HttpServletResponse.SC_OK); // 200
  - response.sendError(HttpServletResponse.SC_NOT_FOUND, "Không tìm thấy trang!");
  - response.setContentType("text/html; charset=UTF-8");
  - response.setHeader("Cache-Control", "no-cache, no-store, must-revalidate");

Cookie #1

- A cookie is a small piece of data sent by a server to the browser, which the browser stores.
- With each subsequent request, the browser will automatically send the appropriate cookie to the server.
- Cookies are stored on the user's computer (in the browser).
- Example
  - Save login information: select "Remember me" when logging in.
  - Save recently viewed products.
  - Save preferred language (Vietnamese/English).
  - Save theme preference (Dark mode/Light mode).

Cookie #2

- In Java web, a cookie is represented by the `Cookie` class

- Cookie creation syntax:

```
Cookie cookie = new Cookie("name", "value");
```

Cookie #3

```
public class CookieServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {

        Cookie userCookie = new Cookie("username", "john");

        // Set Time-to-Live: 24 hours (in seconds)
        userCookie.setMaxAge(24 * 60 * 60);

        // Only send the cookie when accessing the path /myapp/*
        userCookie.setPath("/myapp");

        // Send a cookie to the client.
        response.addCookie(userCookie);

        // Create another cookie
        Cookie themeCookie = new Cookie("theme", "dark");
        themeCookie.setMaxAge(30 * 24 * 60 * 60); // 30 ngày
        themeCookie.setPath("/");
        response.addCookie(themeCookie);

        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println(“Hello");
    }
}
```

Session

Session #1

- HTTP is a stateless protocol, each request is processed independently.
- The server retains no memory of previous requests from the same client.
- Example
  - When you successfully log in, how does the server know that the subsequent request is still coming from you?
  - When you add a product to your shopping cart, how does the server know who that cart belongs to?
- The solution is Sessions.

Session #2

- A session is a mechanism that allows the server to store information about a specific client across multiple requests.

- Each session is identified by a unique **Session ID**.

- A session persists throughout the user's interaction with the application (from login until logout or session timeout).

Session #3 – how it works

1. The client (browser) sends the initial request to the server.
2. The server creates a new session and assigns it a Session ID.
3. The server sends this Session ID back to the client (usually via a cookie).
4. In subsequent requests, the client includes the Session ID.
5. The server uses the Session ID to locate and retrieve the corresponding session data.

HttpSession

- https://tomcat.apache.org/tomcat-5.5-doc/servletapi/index.html

- In Java web, a session is represented by the `HttpSession` interface.

- Example
  - Get current session:
    ```
    HttpSession session = request.getSession();
    ```
  - Retrieve the current session; return null if it does not exist (do not create a new one):
    ```
    HttpSession session = request.getSession(false);
    ```
  - Terminate session: `session.invalidate();`

Cookie vs Session

- Cookie is stored on client's web browser.
  - Used for storing small, non-sensitive data (e.g., preferences, themes).

- Session is stored on server side.
  - Used for storing sensitive/important data (e.g., user authentication, shopping cart).

Q&A – Thank you